

No	Contents
22	Explain the concept of authorization in middleware & routes in Next.js?
23	The difference between <code>use server</code> and <code>use client</code> in Next.js?
24	Understand the concept of server actions in Next.js?
25	Whats are the benifit of using server actions in Next.js?
26	Whats are the problem of using server actions in Next.js?
27	Alternative options instead of server actions in Next.js?
28	Alternative solutions example of not using server actions in Next.js?
29	JWT Token in Next.js App Router?
30	Context of JWT Token in Next.js App Router?
31	Is App Router better than Pages Router in Next.js?
32	How to handle global state management in Next.js with the App Router?
33	What is the fetch API in Next.js App Router?
34	How do you create route groups in the App Router?
35	What are parallel routes in Next.js App Router?
36	How do you implement intercepting routes in App Router?
37	What is the loading.js file in App Router?
38	How do you handle not-found pages in App Router?
39	What is the template.js file in App Router?
40	How do you implement nested layouts in App Router?
41	What are route handlers vs API routes in App Router?
42	How do you handle streaming and suspense in App Router?
43	What is React Server Components (RSC) in App Router?
44	How do you handle error boundaries in App Router?
45	How do you differentiate between server and client components in Next.js?
46	How do you handle internationalization (i18n) in Next.js with the App Router?
47	What is use server, why and when to use it in Next.js?
48	What are the best practices of nextjs api routes?
49	How to validate and sanitize input data in Next.js API routes?
50	How to use proper HTTP methods in Next.js API routes?

Common

[↑ Back to Top](#)

1. What is NextJS?

Next.js is a React framework for building full-stack web applications.

[↑ Back to Top](#)

2. How do you create a new Next.js project?

Using command

```
| npx create-next-app@latest
```

[↑ Back to Top](#)

3. What is the purpose of the `pages` or `app` directory in Next.js?

It contains React components that are automatically routed based on their file name.

[↑ Back to Top](#)

4. What is file based routing in Next.js?

Routing based on the file structure in the `pages` or `app` directory.

[↑ Back to Top](#)

5. What are the key features of Next.js?

- Server Side Rendering (SSR): Next.js allows rendering React components on the server before sending them to the client, improving performance and SEO.
- Static Site Generation (SSG): It pre-renders pages at build time, useful for blogs or e-commerce sites.
- API Routes: You can build a backend using API routes in the same codebase without needing an external server.
- File Based Routing: Next.js automatically creates routes based on the file structure inside the `pages` directory.
- Client Side Rendering (CSR): Like React, Next.js also supports traditional client-side rendering.
- Incremental Side Rendering:
- Image Optimization: Built-in image optimization capabilities that reduce image sizes and enhance loading times.
- Automatic Code Splitting: Next.js splits the code into smaller bundles, which are loaded only when required, improving performance.
- TypeScript Support: Native support for TypeScript, enabling strict typing and better developer experience.
- Incremental Static Regeneration (ISR): Pages can be statically generated at runtime and updated incrementally.
- Fast Refresh: Provides an instant feedback loop while coding, similar to React's hot reloading.

[↑ Back to Top](#)

6. What are the differences between Next.js and React.js

Feature	Next.js	React.js
Rendering	Supports Server-Side Rendering (SSR), Static Site Generation (SSG), and Client-Side Rendering (CSR).	Only supports Client-Side Rendering (CSR) by default.
Routing	Built-in file-based routing system. Automatically generates routes based on the folder structure.	No built-in routing. Requires libraries like React Router.
SEO	Excellent for SEO as it supports SSR and SSG, allowing pre-rendered content to be indexed by search engines.	Limited SEO capabilities due to client-side rendering. Additional work is needed for SEO optimization.
Performance	Faster initial page load due to SSR, automatic code splitting, and static generation.	May have slower page loads for large apps since everything is rendered on the client.
Configuration	Minimal configuration required. Comes with SSR, SSG, and routing out of the box.	Requires manual setup for SSR, routing, and other advanced features.
Learning Curve	Slightly steeper due to built-in advanced features like SSR, SSG, and API routes.	Easier to learn initially, but requires additional tools for SSR and routing.
API Routes	Built-in API routes that can handle backend logic within the same project.	No support for API routes; requires external tools for backend development.
Code Splitting	Automatically splits code into smaller bundles, loading only what's needed for a specific page.	Requires manual code splitting or use of lazy loading to optimize performance.
Deployment	Optimized for easy deployment on platforms like Vercel (creators of Next.js) and supports serverless functions.	Deployment typically requires additional configuration for optimized hosting and SSR.
Image Optimization	Has a built-in Image component for automatic image resizing and optimization.	Does not provide image optimization; developers need third-party libraries for that.

[↑ Back to Top](#)

7. What is the difference between client-side and server-side rendering in Next.js?

Client-side rendering (CSR) means that the browser fetches the JavaScript and renders the page on the client side, while server-side rendering (SSR) means that the server generates the HTML and sends it to the client.

[↑ Back to Top](#)

8. What is the Link component in Next.js?

A component for client side navigation between pages.

```
import Link from "next/link";

<Link href="/">Home</Link>
<Link href="/about">About</Link>
```

[↑ Back to Top](#)

9. What is the useRouter hook in Next.js?

A hook that allows access to the router object and perform navigation. The `useRouter` hook allows you to programmatically change routes inside `client` components.

[↑ Back to Top](#)

10. What is the difference between push and replace in useRouter?

The `push` method adds a new entry to the browser's history stack, while `replace` replaces the current entry in the history stack.

```
const router = useRouter();

// Pushes a new route
router.push("/new-route");

// Replaces the current route
router.replace("/new-route");
```

[↑ Back to Top](#)

11. How do you navigate programmatically in Next.js?

Using `useRouter()` hook.

```
const router = useRouter();

function handleClick() {
  router.push(`/path`);
}

<button onClick={handleClick}>Go There</button>;
```



```
router.push(href: string, { scroll: boolean })
```

[↑ Back to Top](#)

12. How do you enable TypeScript in a Next.js project?

By adding a `tsconfig.json` file.

[↑ Back to Top](#)

13. What is API Routes in Next.js?

A feature to create API endpoints in the `pages/api` or `app/api` directory. It allow you to create custom request handlers for a given route using the Web Request and Response APIs.

[↑ Back to Top](#)

14. What is the public folder in Next.js?

A folder for static assets to be served from the root URL.

```
public/
├─ favicon.ico
├─ robots.txt
├─ images/
└─ profile.jpg
```

[↑ Back to Top](#)

15. What is dynamic import in Next.js?

A feature to load components or modules dynamically.

```
const ComponentA = dynamic(() => import("../components/A"));
const ComponentB = dynamic(() => import("../components/B"));
```

[↑ Back to Top](#)

16. How do you handle environment variables in Next.js?

By adding them to .env.local and accessing via process.env.

[↑ Back to Top](#)

17. What is the default port for a Next.js app?

Port 3000.

[↑ Back to Top](#)

18. How to change default port for a Next.js app?

```
"scripts": {
  "dev": "next dev -p 8080", // for dev
  "start": "next start -p 8080" // for prod
},
```

[↑ Back to Top](#)

19. What is Fast Refresh in Next.js?

A feature for quick feedback when editing React components.

[↑ Back to Top](#)

20. What is next.config.js?

A configuration file to customize Next.js settings.

```
// @ts-check

/** @type {import('next').NextConfig} */
const nextConfig = {
  /* config options here */
};

module.exports = nextConfig;
```

[↑ Back to Top](#)

21. How do you add component-level CSS in Next.js?

Using CSS modules with a `.module.css` file extension.

```
// styles.module.css
.example {
  color: red;
}

// Component.js
```



```
import styles from './styles.module.css';

export default function Component() {
  return <div className={styles.example}>Hello World!</div>;
}
```

[↑ Back to Top](#)

22. How do you add global CSS in Next.js?

By importing CSS files in the _app.js file.

[↑ Back to Top](#)

23. How do you use Tailwind CSS in Next.js?

By installing Tailwind CSS and configuring it in the next.config.js file.

```
npm install tailwindcss postcss autoprefixer
npx tailwindcss init -p
```



Then, add the following to your `tailwind.config.js` :

```
module.exports = {
  content: [
    './pages/**/*.{js,ts,jsx,tsx}',
    './components/**/*.{js,ts,jsx,tsx}',
  ],
  theme: {
    extend: {},
  },
  plugins: [],
};
```



And import Tailwind CSS in your _app.js:

```
import "tailwindcss/tailwind.css";
```



[↑ Back to Top](#)

24. What is server side rendering (SSR) in Next.js?

Rendering pages on each request. If a page uses Server-side Rendering, the page HTML is generated on each request.

```
export async function getServerSideProps() {
  const res = await fetch("https://api.github.com/repos/vercel/next.js");
  const repo = await res.json();
  return { props: { repo } };
}

export default function Page({ repo }) {
  return <p>{repo.stargazers_count} Stars</p>;
}
```



[↑ Back to Top](#)

25. What is static site generation (SSG) in Next.js?

Pre-rendering pages at build time. If a page uses Static Generation, the page HTML is generated at build time.

```
export async function getStaticProps() {
  const res = await fetch("https://api.github.com/repos/vercel/next.js");
  const repo = await res.json();
  return { props: { repo } };
}

export default function Page({ repo }) {
  return <p>{repo.stargazers_count} Stars</p>;
}
```



[↑ Back to Top](#)

26. What is the difference between static site generation and server side rendering?

Static site generation (SSG) pre-renders at build time, server side rendering (SSR) pre-renders on each request.

[↑ Back to Top](#)

27. What is pre-rendering in Next.js?

Generating HTML for pages in advance, instead of on each request.

```
export async function getStaticProps() {
  const res = await fetch("https://api.github.com/repos/vercel/next.js");
  const repo = await res.json();
  return { props: { repo } };
}

export default function Page({ repo }) {
  return <p>{repo.stargazers_count} Stars</p>;
}
```

[↑ Back to Top](#)

28. What is incremental static regeneration (ISR) in Next.js?

Incremental Static Regeneration is a technique in Next.js that allows you to update static pages at runtime without rebuilding the entire site. This feature introduces a seamless way to serve both static and dynamic content by revalidating and regenerating pages in the background.

```
export async function getStaticProps() {
  const res = await fetch("https://api.github.com/repos/vercel/next.js");
  const repo = await res.json();
  return { props: { repo }, revalidate: 1 };
}
```

[↑ Back to Top](#)

29. What is the Image component in Next.js?

A component that optimizes images for faster loading.

```
export default function Page() {
  return (
    <Image
      src={profilePic}
      alt="Picture of the author"
      // width={500} automatically provided
      // height={500} automatically provided
      // blurDataURL="data:..." automatically provided
      // placeholder="blur" // Optional blur-up while loading
    />
  );
}
```

[↑ Back to Top](#)

30. How do you deploy a Next.js app to Vercel?

By connecting the git/github repository to Vercel and deploying it.

[↑ Back to Top](#)

31. How do you handle redirects in Next.js?

There are a few ways you can handle redirects in Next.js. One of them is by configuring redirects in next.config.js.

```
module.exports = {
  async redirects() {
    return [
```

```
{
  source: "/about",
  destination: "/about-us",
  permanent: true,
},
];
},
};
```

[↑ Back to Top](#)

32. What is the Head component in Next.js?

A component for modifying the of a page.

```
import Head from "next/head";

<Head>
  <title>My page title</title>
  <meta name="viewport" content="initial-scale=1.0, width=device-width" />
</Head>;
```

[↑ Back to Top](#)

33. What is the next/head package used for?

To manage the document head for meta tags, title,description, og etc.

```
import Head from "next/head";

export default function Home() {
  return (
    <div>
      <Head>
        <title>My page title</title>
        <meta name="description" content="My description" />
      </Head>
      <h1>Hello World!</h1>
    </div>
  );
}
```

[↑ Back to Top](#)

34. How do you add custom headers in Next.js?

By configuring headers in next.config.js.

[↑ Back to Top](#)

35. What is the use of next export command?

To export a static version of the Next.js app.

[↑ Back to Top](#)

36. How do you optimize fonts in Next.js?

By using the built-in font optimization feature.

[↑ Back to Top](#)

37. How do you enable custom fonts in Next.js?

By using the next/font package to optimize and load custom fonts.

```
import { Inter } from "next/font/google";

const inter = Inter({ subsets: ["latin"] });

export default function Home() {
  return (
```

```
<main className={inter.className}>
  <h1>Hello World!</h1>
</main>
);
}
```

[↑ Back to Top](#)

38. How do you configure Webpack in Next.js?

By adding a custom webpack configuration in next.config.js.

```
module.exports = {
  webpack: (
    config,
    { buildId, dev, isServer, defaultLoaders, webpack }
  ) => {
    // Note: we provide webpack above so you should not `require` it
    // Perform customizations to webpack config
    config.plugins.push(new webpack.IgnorePlugin(/^\/__tests__\/));

    // Important: return the modified config
    return config;
  },
};
```

[↑ Back to Top](#)

39. How do you configure a custom Babel setup in Next.js?

By adding a babel.config.js file.

[↑ Back to Top](#)

40. What is the purpose of next-env.d.ts?

A TypeScript declaration file for Next.js types.

[↑ Back to Top](#)

41. What is the purpose of next-compose-plugins?

To compose and apply multiple Next.js plugins.

[↑ Back to Top](#)

42. How do you add polyfills in Next.js?

By importing them in the _app.js file or using next-polyfill.

[↑ Back to Top](#)

43. What is static optimization in Next.js?

A feature that automatically determines if a page can be statically generated.

[↑ Back to Top](#)

44. How do you handle internationalization (i18n) in Next.js?

By configuring i18n settings in next.config.js.

```
module.exports = {
  i18n: {
    locales: ["en", "fr"],
    defaultLocale: "en",
  },
};
```

[↑ Back to Top](#)

45. What is React Strict Mode in Next.js?

A development mode only feature for highlighting potential problems in an application. It helps to identify unsafe lifecycles, legacy API usage, and a number of other features.

```
module.exports = {  
  reactStrictMode: true,  
};
```



Note: Since Next.js 13.5.1, Strict Mode is true by default with app router, so the above configuration is only necessary for pages. You can still disable Strict Mode by setting `reactStrictMode: false`.

[↑ Back to Top](#)

46. What is a singleton router in Next.js?

A single router instance accessible across the application.

[↑ Back to Top](#)

47. What is next/script used for?

The `next/script` component is used to load external scripts in a Next.js application. It provides features like loading scripts asynchronously, deferring execution, and controlling script loading behavior.

```
import Script from "next/script";  
  
export default function Page() {  
  return (  
    <>  
    <Script src="https://example.com/script.js" strategy="lazyOnload" />  
    <h1>Hello World!</h1>  
    </>  
  );  
}
```



[↑ Back to Top](#)

48. What is middleware?

Middleware allows you to run code before a request is completed. Then, based on the incoming request, you can modify the response by rewriting, redirecting, modifying the request or response headers or responding directly.

```
import { NextResponse } from "next/server";  
  
// This function can be marked `async` if using `await` inside  
export function middleware(request) {  
  return NextResponse.redirect(new URL("/home", request.url));  
}  
  
// See "Matching Paths" below to learn more  
export const config = {  
  matcher: "/about/:path*",  
};
```



[↑ Back to Top](#)

49. What is a custom server in Next.js?

A way to customize the server-side behavior, e.g., with Express.

```
import { createServer } from "http";  
import { parse } from "url";  
import next from "next";  
  
const port = parseInt(process.env.PORT || "3000", 10);  
const dev = process.env.NODE_ENV !== "production";  
const app = next({ dev });  
const handle = app.getRequestHandler();  
  
app.prepare().then(() => {
```



```

createServer((req, res) => {
  const parsedUrl = parse(req.url, true);
  handle(req, res, parsedUrl);
}).listen(port);

console.log(
  `> Server listening at http://localhost:${port} as ${
    dev ? "development" : process.env.NODE_ENV
  }`
);
});

```

[↑ Back to Top](#)

50. How do you perform client-side data fetching in Next.js?

Using `useEffect` and `fetch` or any other data fetching library like `axios`, `fetch` or `swr` by Next.js team.

```

import { useState, useEffect } from "react";

function Profile() {
  const [data, setData] = useState(null);
  const [isLoading, setLoading] = useState(true);

  useEffect(() => {
    fetch("/api/profile")
      .then((res) => res.json())
      .then((data) => {
        setData(data);
        setLoading(false);
      });
  }, []);

  if (isLoading) return <p>Loading...</p>;
  if (!data) return <p>No profile data</p>;

  return (
    <div>
      <h1>{data.name}</h1>
      <p>{data.bio}</p>
    </div>
  );
}

```

[↑ Back to Top](#)

51. How do you set up GraphQL in Next.js?

By installing Apollo Client or any other GraphQL client and configuring it in the `_app.js` file.

```

npm install @apollo/client graphql
yarn add @apollo/client graphql

```

Then, set up Apollo Client in your `_app.js`:

```

import {
  ApolloClient,
  InMemoryCache,
  ApolloProvider,
} from "@apollo/client";

const client = new ApolloClient({
  uri: "https://your-graphql-endpoint.com/graphql",
  cache: new InMemoryCache(),
});

function MyApp({ Component, pageProps }) {
  return (
    <ApolloProvider client={client}>
      <Component {...pageProps} />
    </ApolloProvider>
  );
}

```

```
export default MyApp;
```

[↑ Back to Top](#)

52. How do you create API endpoints in Next.js?

By adding files to the pages/api or app/api directory.

[↑ Back to Top](#)

53. What is the use of next-seo in Next.js?

next-seo is a plugin for managing SEO metadata in Next.js applications, making it easier to set and manage meta tags, Open Graph tags, and other SEO-related elements.

```
import { DefaultSeo } from "next-seo";

function MyApp({ Component, pageProps }) {
  return (
    <>
      <DefaultSeo
        title="My Next.js App"
        description="A description of my Next.js app"
        openGraph={{
          type: "website",
          locale: "en_IE",
          url: "https://www.example.com/",
          site_name: "My Next.js App",
        }}
      />
      <Component {...pageProps} />
    </>
  );
}

export default MyApp;
```

[↑ Back to Top](#)

54. How do you handle routing in a Next.js app?

Using file-based routing in the pages or app directory.

[↑ Back to Top](#)

55. How do you configure next-i18next in Next.js?

By creating a next-i18next.config.js file and initializing it in the app.

```
// next-i18next.config.js
module.exports = {
  i18n: {
    defaultLocale: "en",
    locales: ["en", "fr"],
  },
};
```

Then, initialize it in your app:

```
import { appWithTranslation } from "next-i18next";
import nextI18NextConfig from "../next-i18next.config";

function MyApp({ Component, pageProps }) {
  return <Component {...pageProps} />;
}

export default appWithTranslation(MyApp, nextI18NextConfig);
```

[↑ Back to Top](#)

56. What is `ssr: false` in dynamic import?

It disables server-side rendering for a dynamically imported component, ensuring it only loads on the client side.

```
import dynamic from "next/dynamic";

const DynamicComponent = dynamic(() => import("../components/hello"), {
  ssr: false,
});

function Home() {
  return (
    <section>
      <Header />
      <DynamicComponent />
      <Footer />
    </section>
  );
}

export default Home;
```

[↑ Back to Top](#)

57. How do you add Google Analytics to a Next.js project?

By using the `next/script` component to load the Google Analytics script.

```
import Script from "next/script";

export default function MyApp() {
  return (
    <>
      <Script
        src={`https://www.googletagmanager.com/gtag/js?id=YOUR_TRACKING_ID`}
        strategy="afterInteractive"
      />
      <Script id="google-analytics" strategy="afterInteractive">
        {`
          window.dataLayer = window.dataLayer || [];
          function gtag(){dataLayer.push(arguments);}
          gtag('js', new Date());
          gtag('config', 'YOUR_TRACKING_ID');
        `}
      </Script>
    </>
  );
}
```

[↑ Back to Top](#)

58. How do you add meta tags in Next.js?

Using the `Head` component from `next/head`.

```
import Head from "next/head";

function IndexPage() {
  return (
    <div>
      <Head>
        <title>My page title</title>
        <meta
          name="viewport"
          content="initial-scale=1.0, width=device-width"
        />
        <meta property="og:title" content="My page title" key="title" />
      </Head>
      <p>Hello world!</p>
    </div>
  );
}

export default IndexPage;
```

[↑ Back to Top](#)

59. How to add sitemap in nextjs app?

To add a sitemap in Next.js app router, you can use the `next-sitemap` package. First, install it:

```
npm install next-sitemap
```

Then, create a `next-sitemap.config.js` file in the root of your project and configure your sitemap options.

```
/** @type {import('next-sitemap').IConfig} */
module.exports = {
  siteUrl: process.env.SITE_URL || "https://example.com",
  generateRobotsTxt: true, // (optional)
  changefreq: "daily", // (optional)
  priority: 0.7, // (optional)
  sitemapSize: 7000, // (optional)
  exclude: ["/404", "/500"], // (optional)
  robotsTxtOptions: {
    policies: [
      { userAgent: "*", allow: "/" },
      { userAgent: "Googlebot", disallow: "/private" },
    ],
  },
};
```

Finally, run the following command to generate the sitemap:

```
npx next-sitemap
```

This will create a `sitemap.xml` file in the `public` directory of your Next.js app.

[↑ Back to Top](#)

60. How do you handle CORS in Next.js API routes?

By setting appropriate headers in the API route response.

```
export default function handler(req, res) {
  res.setHeader("Access-Control-Allow-Origin", "*");
  res.setHeader("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE");
  res.setHeader("Access-Control-Allow-Headers", "Content-Type");
  res.status(200).json({ message: "CORS enabled" });
}
```

[↑ Back to Top](#)

61. How do you manage cookies in Next.js?

By using the cookie package or next-cookies to read and write cookies in API routes or server-side functions.

```
import Cookies from "cookies";

export default function handler(req, res) {
  const cookies = new Cookies(req, res);
  cookies.set("token", "value", { httpOnly: true });
  res.status(200).json({ message: "Cookie set" });
}
```

[↑ Back to Top](#)

62. What is the purpose of next/dynamic?

For dynamic importing of components with support for SSR.

```
import dynamic from "next/dynamic";

// Client-side only component
const DynamicComponentWithNoSSR = dynamic(
```

```
() => import("../components/hello"),
{
  ssr: false,
}
);

function Home() {
  return (
    <div>
      <Header />
      <DynamicComponentWithNoSSR />
      <Footer />
    </div>
  );
}

export default Home;
```

[↑ Back to Top](#)

63. How to consider security in nextjs app router?

To consider security in Next.js app router, you can follow these best practices:

- Use HTTPS for all requests to ensure data is encrypted in transit.
- Implement authentication and authorization to protect sensitive routes.
- Sanitize user input to prevent XSS attacks.
- Use environment variables to store sensitive information like API keys.
- Regularly update dependencies to patch known vulnerabilities.
- Implement Content Security Policy (CSP) headers to mitigate XSS attacks.
- Use secure cookies with the `HttpOnly` and `Secure` flags.

[↑ Back to Top](#)

64. What is the useTranslation hook in Next.js?

A hook provided by next-i18next for internationalization.

[↑ Back to Top](#)

65. What is AMP in Next.js?

A framework for creating fast, mobile-friendly pages.

[↑ Back to Top](#)

66. How do you enable AMP in Next.js?

By adding export const config = { amp: true } to a page.

[↑ Back to Top](#)

67. What is the next/image component used for?

For image optimization and responsive images.

[↑ Back to Top](#)

68. What is the next/link component used for?

For client-side navigation between pages.

[↑ Back to Top](#)

69. What is the difference between pages and components directories?

Pages are routes, components are reusable UI elements.

[↑ Back to Top](#)

70. How do you handle static files in Next.js?

By placing them in the `public` directory, which is served at the root URL.

```
public/  
├── images/  
│   └── logo.png  
└── favicon.ico
```



You can access these files using `/images/logo.png` or `/favicon.ico`.

[↑ Back to Top](#)

71. List some common performance optimization techniques in Next.js.

- Use static generation (SSG) for pages that can be pre-rendered.
- Implement incremental static regeneration (ISR) for dynamic content.
- Use the `next/image` component for optimized images.
- Enable code splitting and tree shaking.
- Use dynamic imports for large components.
- Optimize CSS with CSS modules or styled-components.
- Leverage caching strategies for API routes.

[↑ Back to Top](#)

72. Mention some common security practices in Next.js.

- Use HTTPS for secure communication.
- Implement authentication and authorization.
- Sanitize user input to prevent XSS attacks.
- Use environment variables for sensitive data.
- Regularly update dependencies to patch vulnerabilities.
- Implement Content Security Policy (CSP) headers.
- Use secure cookies with `HttpOnly` and `Secure` flags.

[↑ Back to Top](#)

73. Are there any limitations of Next.js?

- Limited support for non-React libraries.
- Requires a Node.js server for server-side rendering.
- Some features may not be compatible with static site generation.
- Learning curve for developers new to React or Next.js.

[↑ Back to Top](#)

74. Is Next.js suitable for large-scale applications?

Yes, Next.js is suitable for large-scale applications due to its features like server-side rendering, static site generation, and API routes. It also supports code splitting, dynamic imports, and incremental static regeneration, which help in managing large codebases efficiently.

[↑ Back to Top](#)

75. How nextjs are full stack framework?

Next.js is considered a full-stack framework because it allows developers to build both the frontend and backend of web applications within a single codebase. It provides features like server-side rendering, static site generation, API routes, and database integration, enabling the development of complete web applications without needing separate frameworks for the frontend and backend.

[↑ Back to Top](#)

76. Prevent API routes from being accessed by the client?

To prevent API routes from being accessed by the client, you can implement authentication and authorization checks in your API route handlers. This ensures that only authenticated users can access the API endpoints.

```
export default function handler(req, res) {  
  const token = req.headers.authorization;
```



```

if (!token || !isValidToken(token)) {
  return res.status(401).json({ error: "Unauthorized" });
}

// Handle the request
res.status(200).json({ message: "Success" });
}

```

[↑ Back to Top](#)

77. JWT Token in Next.js?

JSON Web Tokens (JWT) can be used in Next.js for authentication and authorization. You can create a JWT token upon user login and store it in a cookie or local storage. Then, you can verify the token in API routes or server-side functions to authenticate users.

```

import jwt from "jsonwebtoken";

// Create a JWT token
const token = jwt.sign({ userId: user.id }, process.env.JWT_SECRET, {
  expiresIn: "1h",
});

// Verify the JWT token
jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
  if (err) {
    return res.status(401).json({ error: "Invalid token" });
  }
  // Proceed with authenticated user
});

```

[↑ Back to Top](#)

78. How do you handle global styles in the App Router?

By creating a `globals.css` file in the `app` directory and importing it in the `_app.js` file.

```

/* app/globals.css */
body {
  margin: 0;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto,
    "Helvetica Neue", Arial, sans-serif;
}

```

Then import it in your `_app.js` :

```

// app/_app.js
import "../globals.css";

export default function MyApp({ Component, pageProps }) {
  return <Component {...pageProps} />;
}

```

[↑ Back to Top](#)

Pages Router

[↑ Back to Top](#)

1. What is the Pages Router in Next.js?

The Pages Router is a file-based routing system in Next.js that automatically creates routes based on the file structure inside the `pages` directory.

[↑ Back to Top](#)

2. How do you create a route in the Pages Router?

In Next.js pages router, you create routes by adding files to the `pages` directory:

- `pages/index.js` - the homepage (/)

- `pages/about.js` - the about page (/about)
- `pages/blog/index.js` - the blog index page (/blog)
- `pages/blog/[slug].js` - dynamic blog posts (/blog/:slug)

[↑ Back to Top](#)

3. How do you create a dynamic route in Next.js?

In the pages directory, you can add bracket syntax to create dynamic routes:

```
pages/posts/[id].js → /posts/1, /posts/2, etc.  
pages/[username]/settings.js → /foo/settings, /bar/settings, etc.  
pages/post/[...all].js → /post/2020/id/title, etc.
```



[↑ Back to Top](#)

4. What is catch all segment in Next.js?

A catch-all segment allows you to match multiple segments in a dynamic route. It is defined using `[...param]` syntax.

This allows you to create routes that can match multiple segments, such as `/docs/nextjs`, `/docs/react`, etc.

```
// pages/docs/[...slug].js  
export default function Docs({ params }) {  
  return <div>Docs: {params.slug.join("/")}</div>;  
}
```



[↑ Back to Top](#)

5. What is the `_app.js` file in Next.js?

A special file for initializing pages. It's used for layout, state, or custom error handling.

[↑ Back to Top](#)

6. What is the `_document.js` file in Next.js?

A custom document for augmenting the application's HTML and body tags.

[↑ Back to Top](#)

7. What is the difference between `_app.js` and `_document.js`?

`_app.js` is for page initialization, `_document.js` is for custom document structure.

[↑ Back to Top](#)

8. What is the `_error.js` file in Next.js?

The `_error.js` file is used to create a custom error page for handling errors such as 404 and 500 in Next.js applications.

[↑ Back to Top](#)

9. How do you create a 404 page in Next.js?

By adding a `pages/404.js` file.

[↑ Back to Top](#)

10. How do you fetch data in a Next.js page?

Using `getStaticProps` or `getServerSideProps` in server side.

```
// getStaticProps  
export async function getStaticProps() {  
  const res = await fetch("https://api.github.com/repos/vercel/next.js");  
  const repo = await res.json();  
  return { props: { repo } };  
}
```



```
export default function Page({ repo }) {  
  return repo.stargazers_count;  
}
```

```
// getServerSideProps  
export async function getServerSideProps() {  
  // Fetch data from external API  
  const res = await fetch("https://api.github.com/repos/vercel/next.js");  
  const repo = await res.json();  
  // Pass data to the page via props  
  return { props: { repo } };  
}  
  
export default function Page({ repo }) {  
  return (  
    <main>  
      <p>{repo.stargazers_count}</p>  
    </main>  
  );  
}
```

[↑ Back to Top](#)

11. What is getStaticProps?

A function that runs at build time to fetch data for a page.

```
export async function getStaticProps(context) {  
  const res = await fetch(`https://...`);  
  const data = await res.json();  
  
  if (!data) {  
    return {  
      notFound: true,  
    };  
  }  
  
  return {  
    props: { data }, // will be passed to the page component as props  
  };  
}
```

[↑ Back to Top](#)

12. What is getServerSideProps?

A function that runs on each request to fetch data for a page.

```
export async function getServerSideProps(context) {  
  const res = await fetch(`https://...`);  
  const data = await res.json();  
  
  if (!data) {  
    return {  
      notFound: true,  
    };  
  }  
  
  return {  
    props: { data }, // will be passed to the page component as props  
  };  
}
```

[↑ Back to Top](#)

13. What is the difference between getStaticProps and getServerSideProps?

getStaticProps runs at build time, getServerSideProps runs on each request.

[↑ Back to Top](#)

14. What is getStaticPaths?

A function that specifies dynamic routes to pre-render based on data.

```
export async function getStaticPaths() {
  const res = await fetch("https://.../posts");
  const posts = await res.json();

  // Get the paths we want to pre-render based on posts
  const paths = posts.map((post) => ({
    params: { id: post.id },
  }));

  // We'll pre-render only these paths at build time.
  // { fallback: false } means other routes should 404.
  return { paths, fallback: false };
}
```

[↑ Back to Top](#)

15. What is fallback in getStaticPaths?

Determines how to handle missing paths, with true, false, or 'blocking'.

```
export async function getStaticPaths() {
  const paths = await getAllPostIds();
  return {
    paths,
    fallback: true, // this will enable fallback for all paths which are not generated at build time
  };
}
```

```
export async function getStaticPaths() {
  const paths = await getAllPostIds();
  return {
    paths,
    fallback: false, // this will return 404 for all paths which are not generated at build time
  };
}
```

```
export async function getStaticPaths() {
  const paths = await getAllPostIds();
  return {
    paths,
    fallback: "blocking", // this will return a static page for all paths which are not generated at build time
  };
}
```

[↑ Back to Top](#)

16. How do you handle API routes in Next.js?

By creating files in the `pages/api` directory, which will be treated as API endpoints.

```
// pages/api/hello.js
export default function handler(req, res) {
  res.status(200).json({ name: "John Doe" });
}
```

You can access this API route at `/api/hello`.

[↑ Back to Top](#)

17. How do you handle custom error pages in Next.js?

By creating a `_error.js` file in the `pages` directory.

```
// pages/_error.js
export default function Error({ statusCode }) {
  return (
    <p>
      {statusCode}
    </p>
  );
}
```

```
? `An error ${statusCode} occurred on server`  
: "An error occurred on client"}  
</p>  
);  
}
```

[↑ Back to Top](#)

18. Are there any limitations of the Pages Router?

Yes, the Pages Router has some limitations compared to the App Router, such as:

- Limited support for nested routes and layouts.
- Less flexibility in handling server components.
- No support for React Server Components.

[↑ Back to Top](#)

19. How do you handle authentication in Next.js with the Pages Router?

By using libraries like `next-auth` or implementing custom authentication logic in API routes.

```
// pages/api/auth/[...nextauth].js  
import NextAuth from "next-auth";  
import Providers from "next-auth/providers";  
  
export default NextAuth({  
  providers: [  
    Providers.Google({  
      clientId: process.env.GOOGLE_CLIENT_ID,  
      clientSecret: process.env.GOOGLE_CLIENT_SECRET,  
    }),  
  ],  
  // Add more configuration options as needed  
});
```

[↑ Back to Top](#)

20. How do you handle middleware in Next.js with the Pages Router?

By creating a custom server or using API routes to implement middleware logic.

```
// pages/api/middleware.js  
export default function middleware(req, res, next) {  
  // Custom middleware logic  
  if (req.headers.authorization) {  
    next(); // Proceed to the next handler  
  } else {  
    res.status(401).json({ error: "Unauthorized" });  
  }  
}
```

[↑ Back to Top](#)

21. How do you handle form submissions in Next.js with the Pages Router?

By using client-side form handling or API routes for server-side handling.

```
// pages/contact.js  
export default function Contact() {  
  const handleSubmit = async (e) => {  
    e.preventDefault();  
    const formData = new FormData(e.target);  
    const response = await fetch("/api/contact", {  
      method: "POST",  
      body: formData,  
    });  
    if (response.ok) {  
      alert("Form submitted successfully!");  
    } else {  
      alert("Error submitting form.");  
    }  
  }  
}
```



```

};

return (
  <form onSubmit={handleSubmit}>
    <input name="name" type="text" placeholder="Name" required />
    <input name="email" type="email" placeholder="Email" required />
    <button type="submit">Submit</button>
  </form>
);
}

```

[↑ Back to Top](#)

22. Are there any performance optimizations available in the Pages Router?

Yes, you can use features like static generation (SSG), server-side rendering (SSR), and incremental static regeneration (ISR) to optimize performance in the Pages Router.

- **Static Generation (SSG):** Pre-render pages at build time.
- **Server-Side Rendering (SSR):** Render pages on each request.
- **Incremental Static Regeneration (ISR):** Update static pages after the build.

[↑ Back to Top](#)

23. How do you handle internationalization in Next.js with the Pages Router?

By using the `next-i18next` library or the built-in internationalization features in Next.js.

```

// next.config.js
module.exports = {
  i18n: {
    locales: ["en", "fr"],
    defaultLocale: "en",
  },
};

```

Then, you can use the `useTranslation` hook from `next-i18next` to handle translations in your components.

```

import { useTranslation } from "next-i18next";

export default function Home() {
  const { t } = useTranslation("common");
  return <h1>{t("welcome")}</h1>;
}

```

[↑ Back to Top](#)

24. How do you handle SEO in Next.js with the Pages Router?

By using the `next/head` component to manage meta tags and other SEO-related elements.

```

import Head from "next/head";

export default function Home() {
  return (
    <>
      <Head>
        <title>My Next.js App</title>
        <meta
          name="description"
          content="A description of my Next.js app"
        />
      </Head>
      <h1>Welcome to My Next.js App</h1>
    </>
  );
}

```

[↑ Back to Top](#)

25. How do you handle static assets in Next.js with the Pages Router?

By placing static assets in the `public` directory, which is served at the root URL.

```
public/
├── images/
│   └── logo.png
└── favicon.ico
```

You can access these files using `/images/logo.png` or `/favicon.ico`.

[↑ Back to Top](#)

26. How cache works in Next.js with the Pages Router?

Next.js uses a built-in caching mechanism for static assets and API routes. You can also implement custom caching strategies using HTTP headers or libraries like `next-cache`.

- **Static Assets:** Cached by default with a long cache lifetime.
- **API Routes:** Can be cached using HTTP headers like `Cache-Control`.

```
export default function handler(req, res) {
  res.setHeader("Cache-Control", "public, max-age=3600, immutable");
  res.status(200).json({ message: "Cached response" });
}
```

[↑ Back to Top](#)

27. Cache revalidation in Next.js with the Pages Router?

Cache revalidation can be handled using the `revalidate` option in `getStaticProps` or by setting appropriate HTTP headers in API routes.

- **Using `revalidate`:** Automatically revalidates static pages after a specified time.

```
export async function getStaticProps() {
  const res = await fetch("https://api.example.com/data");
  const data = await res.json();

  return {
    props: { data },
    revalidate: 10, // Revalidate every 10 seconds
  };
}
```

- **Using HTTP Headers:** Set cache control headers in API routes.

```
export default function handler(req, res) {
  res.setHeader("Cache-Control", "s-maxage=10, stale-while-revalidate");
  res.status(200).json({ message: "Revalidated response" });
}
```

[↑ Back to Top](#)

28. Optimizing images in Next.js with the Pages Router?

By using the `next/image` component, which automatically optimizes images for performance.

```
import Image from "next/image";

export default function Home() {
  return (
    <div>
      <h1>My Next.js App</h1>
      <Image
        src="/images/logo.png"
        alt="Logo"
        width={500}
        height={300}
        quality={75}
      />
    </div>
  );
}
```

The `next/image` component provides features like lazy loading, responsive images, and automatic format selection.

[↑ Back to Top](#)

29. When to choose Pages Router over App Router in Next.js?

You might choose the Pages Router over the App Router in the following scenarios:

- When you need a simple file-based routing system without complex nested routes.
- When you prefer the traditional Next.js routing approach.
- When your application does not require advanced features like server components or layouts.

The Pages Router is suitable for smaller applications or when you want to leverage existing knowledge of Next.js routing.

[↑ Back to Top](#)

30. When to choose App Router over Pages Router in Next.js?

You might choose the App Router over the Pages Router in the following scenarios:

- When you need advanced routing capabilities, such as nested routes and layouts.
- When you want to leverage React Server Components for better performance and flexibility.
- When your application requires more complex data fetching strategies.

The App Router is suitable for larger applications or when you want to take advantage of the latest features in Next.js 13.

[↑ Back to Top](#)

App Router

[↑ Back to Top](#)

1. What is the App Router in Next.js?

The App Router is a new routing system introduced in Next.js 13 that allows for more flexible and powerful routing capabilities, including nested routes, layouts, and server components.

[↑ Back to Top](#)

2. How do you create a route in the App Router?

In the App Router, you create routes by adding files to the `app` directory. Each file corresponds to a route, and you can create nested routes by creating subdirectories.

```
app/
├── page.js           // Home page
├── about/
│   └── page.js       // About page
└── blog/
    ├── page.js       // Blog index page
    └── [slug]/
        └── page.js    // Dynamic blog post page
```

[↑ Back to Top](#)

3. How do you create a dynamic route with app router in Next.js?

In the App Router, you create dynamic routes by using square brackets in the file name. For example, to create a dynamic blog post route, you would create a file named `[slug]/page.js` inside the `blog` directory.

```
app/
└── blog/
    └── [slug]/
        └── page.js    // Dynamic blog post page
```

[↑ Back to Top](#)

4. How do you create custom error pages in Next.js?

By creating `error.js` files in `app` directory.

```
// app/error.js
export default function ErrorPage() {
  return <h1>Custom Error Page</h1>;
}
```



[↑ Back to Top](#)

5. How do you handle form submissions in Next.js?

Using client-side form handling or API routes for server-side handling.

[↑ Back to Top](#)

6. How do you handle middleware in Next.js?

By creating a middleware.js file in the app directory.

[↑ Back to Top](#)

7. How do you implement authentication in Next.js?

Using next-auth or a custom authentication solution.

[↑ Back to Top](#)

8. How to add authjs in nextjs app router?

To add authjs in nextjs app router, you can use the `next-auth` package. First, install it:

```
npm install next-auth
```



Then, create a file named `[...nextauth].js` in the `app/api/auth` directory and configure your authentication providers.

```
import NextAuth from "next-auth";
import Providers from "next-auth/providers";

export default NextAuth({
  providers: [
    Providers.Google({
      clientId: process.env.GOOGLE_CLIENT_ID,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET,
    }),
  ],
  // Add more configuration options as needed
  callbacks: {
    async session(session, user) {
      // Add custom session properties here
      return session;
    },
  },
  pages: {
    signIn: "/auth/signin", // Custom sign-in page
    error: "/auth/error", // Error page
  },
  secret: process.env.NEXTAUTH_SECRET, // Required for JWT encryption
  session: {
    jwt: true, // Use JWT for session management
  },
  jwt: {
    secret: process.env.NEXTAUTH_JWT_SECRET, // Required for JWT encryption
  },
  events: {
    signIn: async (message) => {
      // Custom logic after sign-in
      console.log("User signed in:", message);
    },
  },
  debug: process.env.NODE_ENV === "development", // Enable debug mode in development
  theme: {
    colorScheme: "light", // Change to "dark" for dark mode
    brandColor: "#0000FF", // Custom brand color
    logo: "/logo.png", // Custom logo URL
  },
});
```




```
pages: {
  signIn: "/auth/signin", // Custom sign-in page
  signOut: "/auth/signout", // Custom sign-out page
  error: "/auth/error", // Error page
  verifyRequest: "/auth/verify-request", // Verification request page
  newUser: null, // Will disable the new account creation screen
},
});
```

[↑ Back to Top](#)

9. How do you handle authentication tokens in Next.js?

By using cookies or local storage to store authentication tokens.

```
// Example of setting a token in a cookie
import Cookies from "js-cookie";

function setToken(token) {
  Cookies.set("authToken", token, { expires: 7 }); // Expires in 7 days
}

function getToken() {
  return Cookies.get("authToken");
}
```

[↑ Back to Top](#)

10. How to add credentials in nextjs app router?

To add credentials in Next.js app router, you can use the `next-auth` package with the Credentials provider. First, install it:

```
npm install next-auth
```

Then, create a file named `[...nextauth].js` in the `app/api/auth` directory and configure your credentials provider.

```
import NextAuth from "next-auth";
import CredentialsProvider from "next-auth/providers/credentials";

export default NextAuth({
  providers: [
    CredentialsProvider({
      name: "Credentials",
      credentials: {
        username: { label: "Username", type: "text" },
        password: { label: "Password", type: "password" },
      },
      async authorize(credentials) {
        // Add your own logic to validate credentials here
        const user = { id: 1, name: "John Doe" }; // Example user

        if (
          credentials.username === "admin" &&
          credentials.password === "password"
        ) {
          return user; // Return user object if credentials are valid
        } else {
          return null; // Return null if credentials are invalid
        }
      },
    }),
  ],
  pages: {
    signIn: "/auth/signin", // Custom sign-in page
  },
  callbacks: {
    async session(session, user) {
      session.user = user; // Add user object to session
      return session;
    },
  },
  secret: process.env.NEXTAUTH_SECRET, // Required for JWT encryption
  session: {
    jwt: true, // Use JWT for session management
  },
});
```

```

    },
    jwt: {
      secret: process.env.NEXTAUTH_JWT_SECRET, // Required for JWT encryption
    },
  });

```

[↑ Back to Top](#)

11. What is use server in Next.js?

The `use server` directive is used to indicate that a function should be executed on the server side. It allows you to write server-side logic in a component or function that can be called from the client side.

```

"use server";

export async function myServerFunction() {
  // Server-side logic here
  return "Hello from the server!";
}

```

[↑ Back to Top](#)

12. Difference between using & not using use server in Next.js?

Using `use server` : The function is executed on the server side, allowing access to server-side resources and APIs. It can be used to perform operations that require server-side logic, such as database queries or API calls.

Not using `use server` : The function is executed on the client side, meaning it cannot access server-side resources directly. It can only perform operations that are available in the client environment, such as manipulating the DOM or making client-side API calls.

- Example with `use server` :

```

"use server";
export async function fetchData() {
  const response = await fetch("https://api.example.com/data");
  const data = await response.json();
  return data;
}

```

- Example without `use server` :

```

export async function fetchData() {
  const response = await fetch("https://api.example.com/data");
  const data = await response.json();
  return data;
}

```

- Example with `use server` In Component:

```

import { fetchData } from "../path/to/your/file";

export default function MyComponent() {
  const data = await fetchData(); // This will run on the server side
  return <div>{data}</div>;
}

```

- Example without `use server` In Component:

```

import { fetchData } from "../path/to/your/file";
export default function MyComponent() {
  const data = fetchData(); // This will run on the client side
  return <div>{data}</div>;
}

```

[↑ Back to Top](#)

13. How do you handle API routes in Next.js?

By creating files in the `app/api` directory, where each file corresponds to an API endpoint.

```
// app/api/hello/route.js
export async function GET(request) {
  return new Response("Hello, World!");
}
```

[↑ Back to Top](#)

14. How do middleware work in Next.js?

Middleware in Next.js allows you to run code before a request is completed. You can use it to modify the request or response, redirect users, or perform authentication checks.

```
// app/middleware.js
import { NextResponse } from "next/server";
export function middleware(request) {
  // Perform some logic here
  if (request.nextUrl.pathname === "/" ) {
    return NextResponse.redirect(new URL("/home", request.url));
  }
  return NextResponse.next();
}
```



You can also specify which paths the middleware should apply to:

```
// app/middleware.js
export const config = {
  matcher: ["/about/:path*", "/blog/:path*"],
};
```

[↑ Back to Top](#)

15. What is form action in Next.js?

The `formAction` is a special attribute used in Next.js to define the action URL for a form submission. It allows you to specify a server-side function that will handle the form submission.

```
// app/form-example/page.js
"use server";

export async function handleSubmit(formData) {
  const name = formData.get("name");
  console.log("Form submitted with name:", name);
  return { success: true };
}

export default function FormExample() {
  return (
    <form action={handleSubmit}>
      <input type="text" name="name" />
      <button type="submit">Submit</button>
    </form>
  );
}
```



In this example, when the form is submitted, the `handleSubmit` function will be called on the server side with the form data.

[↑ Back to Top](#)

16. How do you handle file uploads in Next.js?

By using the `formData` API in a server action to handle file uploads.

```
// app/upload/page.js
"use server";

export async function handleUpload(formData) {
  const file = formData.get("file");
  // Process the file (e.g., save it to a storage service)
  console.log("File uploaded:", file.name);
  return { success: true };
}
```



```
export default function UploadPage() {
  return (
    <form action={handleUpload} encType="multipart/form-data">
      <input type="file" name="file" />
      <button type="submit">Upload</button>
    </form>
  );
}
```

[↑ Back to Top](#)

17. Mention some common use cases for the App Router in Next.js.

- Creating nested routes with layouts.
- Implementing server-side rendering for dynamic content.
- Handling API routes for backend functionality.
- Managing authentication and authorization flows.
- Building complex applications with shared layouts and components.

[↑ Back to Top](#)

18. One of the main differences between the App Router and Pages Router in Next.js?

The App Router allows for nested routes, layouts, and server components, while the Pages Router uses a flat file structure for routing and does not support nested routes or layouts.

[↑ Back to Top](#)

19. What is the use of the `use client` directive in Next.js?

The `use client` directive is used to indicate that a component should be rendered on the client side. It allows you to write client-side logic in a component that can be executed in the browser.

```
"use client";

export default function ClientComponent() {
  return <div>This component is rendered on the client side.</div>;
}
```

[↑ Back to Top](#)

20. Is it possible to use both App Router and Pages Router in the same Next.js project?

Yes, it is possible to use both App Router and Pages Router in the same Next.js project. You can have the `app` directory for the App Router and the `pages` directory for the Pages Router, allowing you to take advantage of both routing systems.

[↑ Back to Top](#)

21. Are there any limitations of the App Router in Next.js?

Yes, some limitations of the App Router include:

- It is only available in Next.js 13 and later versions.
- It may not support all features available in the Pages Router.
- Some third-party libraries may not be compatible with the App Router.
- Many features from the Pages Router, such as `getStaticProps` and `getServerSideProps`, are not available in the App Router.
- The App Router is still evolving, and some features may change or be added in future releases.

[↑ Back to Top](#)

22. Explain the concept of authorization in middleware & routes in Next.js.

Authorization in middleware and routes in Next.js involves checking if a user has the necessary permissions to access a specific route or perform an action. This can be done by verifying user roles, permissions, or tokens in the middleware function before allowing access to the route.

```
// app/middleware.js
import { NextResponse } from "next/server";

export function middleware(request) {
  const token = request.cookies.get("authToken");

  if (!token) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  // Additional authorization logic can go here

  return NextResponse.next();
}

export const config = {
  matcher: ["/protected/:path*"], // Apply middleware to protected routes
};
```

In this example, the middleware checks for an authentication token in the cookies. If the token is not present, it redirects the user to the login page. If the token is valid, it allows access to the protected routes.

```
// app/api/protected/route.js
import { NextResponse } from "next/server";
import jwt from "jsonwebtoken";

export async function GET(request) {
  const token = request.cookies.get("authToken");

  if (!token) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    // Handle the request for authorized users
    return NextResponse.json({
      message: "Protected data",
      userId: decoded.userId,
    });
  } catch (error) {
    return NextResponse.json({ error: "Invalid token" }, { status: 401 });
  }
}
```

In this example, the API route checks for the authentication token in the request cookies. If the token is not present, it returns a 401 Unauthorized response. If the token is valid, it returns the protected data.

[⬆️ Back to Top](#)

23. The difference between `use server` and `use client` in Next.js?

- `use server`: Indicates that the function should be executed on the server side. It allows you to write server-side logic that can be called from the client side.
- `use client`: Indicates that the component should be rendered on the client side. It allows you to write client-side logic that can be executed in the browser.

```
// Example of use server
"use server";

export async function fetchData() {
  const response = await fetch("https://api.example.com/data");
  const data = await response.json();
  return data;
}

// Example of use client
"use client";

export default function ClientComponent() {
  return <div>This component is rendered on the client side.</div>;
}
```

[↑ Back to Top](#)

24. Understand the concept of server actions in Next.js.

Server actions in Next.js allow you to define functions that can be executed on the server side when a form is submitted or an action is triggered. These functions can handle data processing, database interactions, or any server-side logic.

```
// app/actions/submitForm.js
"use server";

export async function submitForm(formData) {
  const name = formData.get("name");
  console.log("Form submitted with name:", name);
  return { success: true };
}
```

You can then use this action in a form:

```
// app/form/page.js
import { submitForm } from "../actions/submitForm";

export default function FormPage() {
  return (
    <form action={submitForm}>
      <input type="text" name="name" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

[↑ Back to Top](#)

25. Whats are the benifit of using server actions in Next.js?

- **Performance:** Server actions allow you to offload heavy computations or data processing to the server, reducing the load on the client.
- **Security:** Sensitive operations can be performed on the server, preventing exposure of sensitive data or logic to the client.
- **Simplified Data Fetching:** You can fetch data directly in server actions without needing to manage client-side state or effects.
- **Reduced Client Bundle Size:** By moving logic to the server, you can reduce the amount of JavaScript sent to the client, improving load times.

[↑ Back to Top](#)

26. What's are the problem of using server actions in Next.js?

- **Latency:** Server actions can introduce latency since they require a round trip to the server, which may not be ideal for real-time interactions.
- **Complexity:** Managing server actions can add complexity to your application, especially if you have many actions or need to handle different states.
- **Limited Client-Side Interactivity:** Since server actions are executed on the server, they may not provide the same level of interactivity as client-side functions.
- **Debugging Challenges:** Debugging server actions can be more challenging compared to client-side code, as you may not have access to browser developer tools.

[↑ Back to Top](#)

27. Alternative options instead of server actions in Next.js?

- **API Routes:** You can create API routes to handle server-side logic and data fetching, which can be called from the client side.
- **Client-Side Fetching:** Use client-side data fetching methods like `useEffect` or libraries like SWR or React Query to manage data on the client side.
- **Static Site Generation (SSG):** Use SSG for pages that can be pre-rendered at build time, reducing the need for server actions.
- **Server-Side Rendering (SSR):** Use SSR for dynamic pages that require server-side data fetching on each request.

[↑ Back to Top](#)

28. Alternative solutions example of not using server actions in Next.js?

Instead of using server actions, you can use API routes to handle form submissions or data processing. Here's an example:

```
// app/api/submitForm/route.js
export async function POST(request) {
  const formData = await request.formData();
  const name = formData.get("name");
  console.log("Form submitted with name:", name);
  return new Response(JSON.stringify({ success: true }), {
    status: 200,
    headers: { "Content-Type": "application/json" },
  });
}
```

You can then call this API route from a client-side component:

```
// app/form/page.js
export default function FormPage() {
  const handleSubmit = async (event) => {
    event.preventDefault();
    const formData = new FormData(event.target);
    const response = await fetch("/api/submitForm", {
      method: "POST",
      body: formData,
    });
    const result = await response.json();
    console.log(result);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input type="text" name="name" />
      <button type="submit">Submit</button>
    </form>
  );
}
```

[↑ Back to Top](#)

29. JWT Token in Next.js App Router?

JSON Web Tokens (JWT) can be used in the Next.js App Router for authentication and authorization. You can create a JWT token upon user login and store it in a cookie or local storage. Then, you can verify the token in API routes or server-side functions to authenticate users.

```
import jwt from "jsonwebtoken";

// Create a JWT token
const token = jwt.sign({ userId: user.id }, process.env.JWT_SECRET, {
  expiresIn: "1h",
});

// Verify the JWT token
jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
  if (err) {
    return res.status(401).json({ error: "Invalid token" });
  }
  // Proceed with authenticated user
});
```

Using it on application:

```
// app/api/auth/route.js
import { NextResponse } from "next/server";
import jwt from "jsonwebtoken";
export async function POST(request) {
  const { token } = await request.json();
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    return NextResponse.json({ userId: decoded.userId });
  } catch (error) {
    return NextResponse.json({ error: "Invalid token" }, { status: 401 });
  }
}
```

You can also use JWT tokens for protecting API routes:

```
// app/api/protected/route.js
import { NextResponse } from "next/server";
import jwt from "jsonwebtoken";
export async function GET(request) {
  const token = request.cookies.get("authToken");

  if (!token) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    // Handle the request for authorized users
    return NextResponse.json({
      message: "Protected data",
      userId: decoded.userId,
    });
  } catch (error) {
    return NextResponse.json({ error: "Invalid token" }, { status: 401 });
  }
}
```

[↑ Back to Top](#)

30. Context of JWT Token in Next.js App Router?

The context of using JWT tokens in the Next.js App Router is primarily for authentication and authorization purposes. JWT tokens allow you to securely transmit user information between the client and server, enabling you to verify user identity and permissions without needing to store session data on the server.

This approach is particularly useful for stateless applications where you want to maintain user sessions without relying on server-side session storage.

[↑ Back to Top](#)

31. Is App Router better than Pages Router in Next.js?

The App Router offers more flexibility and features compared to the Pages Router, such as nested routes, layouts, and server components. It is designed for building complex applications with shared layouts and components.

However, the choice between App Router and Pages Router depends on your specific use case. If you need simple routing without nested routes or layouts, the Pages Router may be sufficient.

[↑ Back to Top](#)

32. How to handle global state management in Next.js with the App Router?

You can handle global state management in Next.js with the App Router using libraries like Redux, Zustand, or React Context API. These libraries allow you to create a global store that can be accessed from any component in your application.

o Using React Context API:

```
// app/context/GlobalState.js
import { createContext, useContext, useState } from "react";

const GlobalStateContext = createContext();

export function GlobalStateProvider({ children }) {
  const [state, setState] = useState({ user: null });

  return (
    <GlobalStateContext.Provider value={{ state, setState }}>
      {children}
    </GlobalStateContext.Provider>
  );
}

export function useGlobalState() {
  return useContext(GlobalStateContext);
}
```

Then wrap your application with the `GlobalStateProvider` :


```
// app/layout.js
import { GlobalStateProvider } from "../context/GlobalState";

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <GlobalStateProvider>{children}</GlobalStateProvider>
      </body>
    </html>
  );
}
```

Now you can access the global state in any component using the `useGlobalState` hook.

```
// app/page.js
import { useGlobalState } from "../context/GlobalState";

export default function HomePage() {
  const { state, setState } = useGlobalState();

  return (
    <div>
      <h1>Welcome, {state.user ? state.user.name : "Guest"}</h1>
      <button onClick={() => setState({ user: { name: "John Doe" } })}>
        Log In
      </button>
    </div>
  );
}
```

[↑ Back to Top](#)

33. What is the fetch API in Next.js App Router?

In the Next.js App Router, you can use the `fetch` API to make HTTP requests to external APIs or your own API routes. The `fetch` function is available globally in both server and client components.

- Example of using fetch in a server component:

```
// app/api/data/route.js
export async function GET() {
  const response = await fetch("https://api.example.com/data");
  const data = await response.json();
  return new Response(JSON.stringify(data), {
    headers: { "Content-Type": "application/json" },
  });
}
```

- Example of using fetch in a client component:

```
// app/page.js
import { useEffect, useState } from "react";

export default function HomePage() {
  const [data, setData] = useState(null);

  useEffect(() => {
    async function fetchData() {
      const response = await fetch("/api/data");
      const result = await response.json();
      setData(result);
    }
    fetchData();
  }, []);

  return (
    <div>
      <h1>Data from API</h1>
      <pre>{JSON.stringify(data, null, 2)}</pre>
    </div>
  );
}
```

[↑ Back to Top](#)

34. How do you create route groups in the App Router?

Route groups allow you to organize routes without affecting the URL structure. You create them by wrapping folder names in parentheses `()`.

```
app/
├── (marketing)/
│   ├── about/
│   │   └── page.js    // /about
│   └── contact/
│       └── page.js    // /contact
├── (shop)/
│   ├── products/
│   │   └── page.js    // /products
│   └── cart/
│       └── page.js    // /cart
└── layout.js          // Shared layout
```

Route groups are useful for organizing code, creating different layouts for different sections, or conditionally including layouts.

[↑ Back to Top](#)

35. What are parallel routes in Next.js App Router?

Parallel routes allow you to render multiple pages simultaneously in the same layout. They are defined using slots with the `@` convention.

```
app/
├── layout.js
├── page.js
├── @analytics/
│   └── page.js
└── @team/
    └── page.js
```

```
// app/layout.js
export default function Layout({ children, analytics, team }) {
  return (
    <div>
      <div>{children}</div>
      <div>{analytics}</div>
      <div>{team}</div>
    </div>
  );
}
```

This allows you to render independent pages that can load at different speeds and handle their own loading and error states.

[↑ Back to Top](#)

36. How do you implement intercepting routes in App Router?

Intercepting routes allow you to load a route from another part of your application while keeping the context of the current page, similar to modals.

```
app/
├── feed/
│   └── page.js
├── photo/
│   └── [id]/
│       └── page.js
└── @modal/
    └── (..)photo/
        └── [id]/
            └── page.js
```

The `(..)` convention indicates that you want to intercept routes at the same level. Intercepting routes use conventions like:

- `(.)` - match segments on the same level
- `(..)` - match segments one level above

- `(..)(..)` - match segments two levels above
- `(...)` - match segments from the root app directory

[↑ Back to Top](#)

37. What is the `loading.js` file in App Router?

The `loading.js` file creates loading UI that shows instantly while route segments are loading. It automatically wraps the page and its children in a React Suspense boundary.

```
// app/dashboard/loading.js
export default function Loading() {
  return (
    <div className="loading">
      <p>Loading dashboard...</p>
      <div className="spinner"></div>
    </div>
  );
}
```

```
app/
├── dashboard/
│   ├── loading.js    // Loading UI for dashboard
│   ├── page.js
│   └── settings/
│       ├── loading.js // Loading UI for settings
│       └── page.js
```

The loading UI will be shown immediately on navigation and can be nested for granular loading states.

[↑ Back to Top](#)

38. How do you handle not-found pages in App Router?

You can create custom not-found pages using the `not-found.js` file. This file defines UI to render when the `notFound()` function is thrown within a route segment.

```
// app/not-found.js
import Link from "next/link";

export default function NotFound() {
  return (
    <div>
      <h2>Not Found</h2>
      <p>Could not find requested resource</p>
      <Link href="/">Return Home</Link>
    </div>
  );
}
```

You can also trigger the not-found page programmatically:

```
// app/page.js
import { notFound } from "next/navigation";

export default function Page({ params }) {
  const post = getPost(params.id);

  if (!post) {
    notFound();
  }

  return <div>{post.title}</div>;
}
```

[↑ Back to Top](#)

39. What is the `template.js` file in App Router?

The `template.js` file is similar to `layout.js` but creates a new instance for each of its children on navigation. This means state is not preserved and effects are re-synchronized.

```
// app/template.js
export default function Template({ children }) {
  return <div className="template-wrapper">{children}</div>;
}
```

Key differences from layout:

- **Layout:** State is preserved, DOM elements are not re-created
- **Template:** New instance on navigation, DOM elements are re-created

Templates are useful when you need:

- CSS/JS animations on route changes
- Features that rely on `useEffect` and `useState`
- To change the default browser behavior

[↑ Back to Top](#)

40. How do you implement nested layouts in App Router?

Nested layouts are implemented by creating `layout.js` files in different route segments. Layouts are nested automatically based on the folder structure.

```
app/
├─ layout.js           // Root layout
├─ page.js             // Home page
└─ dashboard/
   ├─ layout.js        // Dashboard layout
   ├─ page.js          // Dashboard page
   └─ settings/
      ├─ layout.js     // Settings layout
      └─ page.js       // Settings page
```

```
// app/layout.js (Root Layout)
export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <nav>Global Navigation</nav>
        {children}
      </body>
    </html>
  );
}

// app/dashboard/layout.js
export default function DashboardLayout({ children }) {
  return (
    <div className="dashboard">
      <aside>Dashboard Sidebar</aside>
      <main>{children}</main>
    </div>
  );
}
```

When visiting `/dashboard/settings`, all three layouts (root, dashboard, settings) will be rendered in a nested structure.

[↑ Back to Top](#)

41. What are route handlers vs API routes in App Router?

In the App Router, API routes are now called "Route Handlers" and use the `route.js` file convention instead of the pages-based approach.

Pages Router (API Routes):

```
pages/api/users.js
```

App Router (Route Handlers):

app/api/users/route.js

```
// app/api/users/route.js
export async function GET(request) {
  const users = await getUsers();
  return Response.json(users);
}

export async function POST(request) {
  const data = await request.json();
  const user = await createUser(data);
  return Response.json(user, { status: 201 });
}

export async function PUT(request) {
  const data = await request.json();
  const user = await updateUser(data);
  return Response.json(user);
}

export async function DELETE(request) {
  await deleteUser(request.nextUrl.searchParams.get("id"));
  return new Response(null, { status: 204 });
}
```

Route handlers support all HTTP methods and provide better TypeScript support and Web APIs compatibility.

[⬆️ Back to Top](#)

42. How do you handle streaming and suspense in App Router?

The App Router has built-in support for streaming and React Suspense, allowing you to progressively render and stream UI to the client.

```
// app/page.js
import { Suspense } from "react";

async function UserProfile({ userId }) {
  const user = await getUserData(userId); // This can be slow
  return <div>Welcome, {user.name}!</div>;
}

async function UserPosts({ userId }) {
  const posts = await getUserPosts(userId); // This can also be slow
  return (
    <div>
      {posts.map((post) => (
        <div key={post.id}>{post.title}</div>
      ))}
    </div>
  );
}

export default function Page({ params }) {
  return (
    <div>
      <h1>User Dashboard</h1>
      <Suspense fallback=<div>Loading profile...</div>>
        <UserProfile userId={params.id} />
      </Suspense>
      <Suspense fallback=<div>Loading posts...</div>>
        <UserPosts userId={params.id} />
      </Suspense>
    </div>
  );
}
```

Benefits of streaming:

- Faster initial page load
- Better perceived performance
- Progressive enhancement

- SEO-friendly as search engines can index content as it streams

[↑ Back to Top](#)

43. What is React Server Components (RSC) in App Router?

React Server Components (RSC) are a new React feature that allows components to be rendered on the server. In the App Router, components are Server Components by default.

Server Components (default):

```
// app/page.js - This is a Server Component
async function ServerComponent() {
  const data = await fetch("https://api.example.com/data");
  const result = await data.json();

  return (
    <div>
      <h1>Server Rendered Data</h1>
      <p>{result.message}</p>
    </div>
  );
}
```

Client Components (opt-in with "use client"):

```
// app/components/ClientComponent.js
"use client";

import { useState } from "react";

export default function ClientComponent() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>Count: {count}</button>
  );
}
```

Benefits of Server Components:

- Direct access to server-side resources (databases, file system)
- No JavaScript bundle sent to client
- Improved performance and SEO
- Automatic code splitting

When to use Client Components:

- Interactive features (event handlers, state)
- Browser-only APIs (localStorage, geolocation)
- React hooks (useState, useEffect)

[↑ Back to Top](#)

44. How do you handle error boundaries in App Router?

You can create error boundaries using the `error.js` file in a route segment. This file defines UI to render when an error is thrown within that segment.

```
// app/error.js
import Link from "next/link";

export default function Error({ error, reset }) {
  return (
    <div>
      <h2>Something went wrong!</h2>
      <p>{error.message}</p>
      <button onClick={reset}>Try Again</button>
      <Link href="/">Return Home</Link>
    </div>
  );
}
```

```
);  
}
```

You can also throw errors programmatically:

```
// app/page.js  
export default function Page() {  
  throw new Error("An unexpected error occurred!");  
  return <div>This will not be rendered.</div>;  
}
```

The error boundary will catch the error and display the custom error UI defined in `error.js`.

[↑ Back to Top](#)

45. How do you differentiate between server and client components in Next.js?

In Next.js, components are Server Components by default. To differentiate and create Client Components, you need to add the `"use client"` directive at the top of the component file.

- **Server Components:** These run on the server and can access server-side resources like databases and file systems. They do not include any client-side JavaScript in the bundle.

```
// app/page.js - This is a Server Component  
async function ServerComponent() {  
  const data = await fetch("https://api.example.com/data");  
  const result = await data.json();  
  
  return (  
    <div>  
      <h1>Server Rendered Data</h1>  
      <p>{result.message}</p>  
    </div>  
  );  
}
```

- **Client Components:** These run on the client side and can use React hooks, manage state, and handle user interactions. They must include the `"use client"` directive.

```
// app/components/ClientComponent.js  
"use client";  
  
import { useState } from "react";  
  
export default function ClientComponent() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <button onClick={() => setCount(count + 1)}>Count: {count}</button>  
  );  
}
```

Use Server Components for static content and data fetching, and Client Components for interactivity and state management.

[↑ Back to Top](#)

46. How do you handle internationalization (i18n) in Next.js with the App Router?

Next.js provides built-in support for internationalization (i18n) in the App Router. You can configure i18n settings in the `next.config.js` file.

```
// next.config.js  
module.exports = {  
  i18n: {  
    locales: ["en", "fr", "de"],  
    defaultLocale: "en",  
  },  
};
```

You can then create localized content by using dynamic route segments for different languages.

```

app/
├── [locale]/
│   ├── page.js      // Localized home page
│   └── about/
│       └── page.js  // Localized about page

```

```

// app/[locale]/page.js
import { useRouter } from "next/router";

export default function HomePage() {
  const { locale } = useRouter();

  return <div>Welcome to the {locale} version of the site!</div>;
}

```

You can also use libraries like `next-translate` or `react-i18next` for more advanced i18n features.

[↑ Back to Top](#)

47. What is use server, why and when to use it in Next.js?

The `"use server"` directive in Next.js is used to indicate that a function should be executed on the server side. It allows you to write server-side logic that can be called from the client side, such as handling form submissions or processing data.

You should use `"use server"` when you need to perform operations that require server-side capabilities, such as:

- Accessing databases or file systems
- Performing secure operations that should not be exposed to the client
- Handling form submissions and processing data on the server

Example usage:

```

// app/form-example/page.js
"use server";

export async function handleSubmit(formData) {
  const name = formData.get("name");
  console.log("Form submitted with name:", name);
  return { success: true };
}

export default function FormExample() {
  return (
    <form action={handleSubmit}>
      <input type="text" name="name" />
      <button type="submit">Submit</button>
    </form>
  );
}

```

In this example, the `handleSubmit` function is marked with `"use server"`, indicating that it will run on the server when the form is submitted.

[↑ Back to Top](#)

48. What are the best practices of nextjs api routes?

- Use proper HTTP methods (GET, POST, PUT, DELETE) for different operations.
- Validate and sanitize input data to prevent security vulnerabilities.
- Handle errors gracefully and return appropriate HTTP status codes.
- Use middleware for common tasks like authentication and logging.
- Keep API routes modular and organized in separate files or folders.
- Optimize performance by caching responses when appropriate.
- Document your API endpoints for easier consumption by other developers.

[↑ Back to Top](#)

49. How to use proper HTTP methods in Next.js API routes?

In Next.js API routes, you can define different functions for each HTTP method (GET, POST, PUT, DELETE) within the same route file. This allows you to handle different types of requests appropriately.

Example:

```
// app/api/users/route.js
export async function GET(request) {
  const users = await getUsers();
  return new Response(JSON.stringify(users), {
    headers: { "Content-Type": "application/json" },
  });
}

export async function POST(request) {
  const data = await request.json();
  const user = await createUser(data);
  return new Response(JSON.stringify(user), {
    status: 201,
    headers: { "Content-Type": "application/json" },
  });
}

export async function PUT(request) {
  const data = await request.json();
  const user = await updateUser(data);
  return new Response(JSON.stringify(user), {
    headers: { "Content-Type": "application/json" },
  });
}

export async function DELETE(request) {
  const { searchParams } = new URL(request.url);
  const userId = searchParams.get("id");
  await deleteUser(userId);
  return new Response(null, { status: 204 });
}
```

In this example, each function corresponds to a specific HTTP method, allowing you to handle requests accordingly.

[↑ Back to Top](#)

50. How to validate and sanitize input data in Next.js API routes?

To validate and sanitize input data in Next.js API routes, you can use libraries like `Joi`, `Yup`, or `validator.js`. These libraries help ensure that the data received from clients meets the expected format and is safe to use.

Example using `Joi`:

```
// app/api/users/route.js
import Joi from "joi";

const userSchema = Joi.object({
```

Releases

No releases published

Packages

No packages published